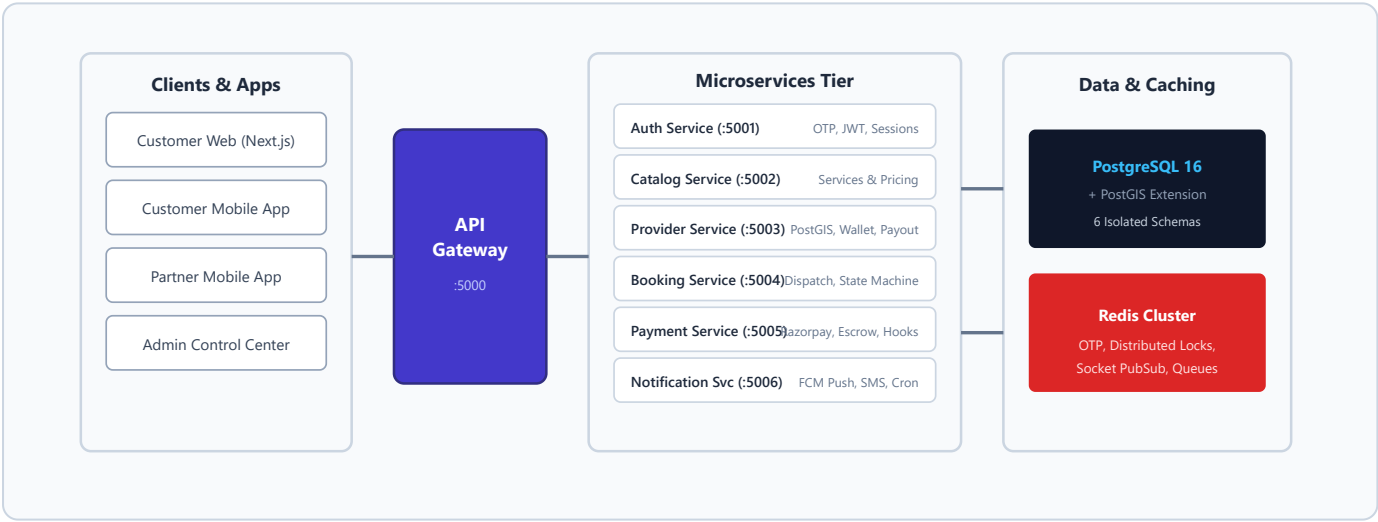
BharatClap Technical Architecture

Complete End-to-End System, Security, Data, Dispatch, and Financial Engine Specification

ARCHITECTURE	PRIMARY DATABASE	PAYMENT GATEWAY	CACHE / MESSAGE BROKER
Event-Driven Microservices	PostgreSQL 16 + PostGIS	Razorpay / RazorpayX Payouts	Redis / BullMQ / Socket.io

1. Executive Summary & Topology

BharatClap is an on-demand home and local services marketplace (similar to Urban Company). The platform operates on a decoupled, resilient microservices architecture orchestrated via an API Gateway. Key capabilities include real-time geospatial partner dispatching, double-entry wallet accounting, escrow-protected booking lifecycle, and automated bank settlements via RazorpayX.



2. Authentication & Identity Architecture

The `auth-service` centralizes identity management for customers, service partners, and administrators. The authentication workflow eliminates password vulnerabilities through **Phone OTP authentication** with strict rate-limiting, short-lived JWTs, and refresh token rotation.

> **Zero-Trust Token Rotation (RFC 6749 Compliant)**
>
> Every refresh cycle produces a new pair of access and refresh tokens. If a previously consumed refresh token is presented, the system detects a token reuse attack, revokes the entire refresh token family, and terminates the active session immediately.

TOKEN / CREDENTIAL	LIFESPAN	STORAGE LOCATION	SECURITY MECHANISM
Access Token	15 Minutes	Client Memory / Authorization Header	RS256/HS256 JWT, verified statelessly by API Gateway.
Refresh Token	30 Days	`auth.refresh_tokens` table	Cryptographic SHA-256 hash stored. Revoked on rotation.
Phone OTP	5 Minutes	Redis (TTL 300s)	Max 3 failed attempts before lock. Rate-limited per IP/phone.
Partner Bank Info	Persistent	`provider.providers` (encrypted)	AES-256 GCM encrypted at rest (account number & IFSC).

Login & Verification Sequence

1. **OTP Request (`POST /api/auth/send-otp`)**
   Gateway applies IP rate limiting; Auth Service generates a cryptographically secure 6-digit OTP, stores it in Redis with a 5-minute TTL, and dispatches via SMS provider (Fast2SMS / Twilio).

2. **OTP Validation (`POST /api/auth/verify-otp`)**
   Auth Service checks Redis. On match, retrieves or creates the user in `auth.users`, links user device info, and records an active session in `auth.sessions`.

3. **RBAC & Gateway Context Propagation**
   Gateway validates the JWT and injects trusted internal headers (`x-user-id` , `x-user-role` , `x-session-id`) to downstream services, blocking direct unauthenticated external access.

3. Database Architecture & Multi-Schema Isolation

The database engine is PostgreSQL 16 enriched with **PostGIS** for spatial calculations. To support domain-driven microservices while retaining relational integrity, tables are grouped into isolated schemas:

SCHEMA	KEY TABLES	ENGINE / EXTENSIONS	ARCHITECTURAL RESPONSIBILITY
`auth`	`users`, `sessions`, `refresh_tokens`, `user_addresses`	B-Tree Indexes, GIN	Profiles, addresses, authentication audit trails.
`catalog`	`categories`, `services`, `subservices`, `pricing_rules`	Tree Hierarchies	Service directory, dynamic pricing modifiers.
`booking`	`bookings`, `booking_items`, `booking_event_outbox`	Transactional Outbox	Booking state machine, scheduling, event journal.
`provider`	`providers`, `job_requests`, `wallet_ledger`, `provider_settlements`	PostGIS (GiST Index)	Partner live GPS, spatial dispatch, wallet ledger.
`payment`	`payments`, `payment_orders`, `payment_webhooks`	Idempotency Unique Keys	Razorpay escrow capture, webhook processing.
`accounting`	`journal_entries`, `ledger_accounts`	Immutable Append-Only	Platform revenue, escrow liability, double-entry audit.

> **Transactional Outbox Pattern**
>
> To prevent dual-write inconsistencies between PostgreSQL and external message queues or third-party webhooks, events are written directly inside the database transaction (e.g., `booking.booking_event_outbox`). Background workers continuously poll and publish guaranteed events.

4. Order Creation & Geospatial Allocation Engine

Order allocation is handled cooperatively between `booking-service` and `provider-service`. Once a booking is created, it enters the **3-Tier Geospatial Dispatch Scoring Engine**:



3-Tier Fallback Hierarchy

TIER	CRITERIA	TIMEOUT	FALLBACK TRIGGER
TIER 1: INSTANT	Verified, GPS updated within 10m, `is_online = true`, distance ≤ 30km, no schedule clash.	90 Seconds	No online partners or offer expired without acceptance.
TIER 2: NEARBY	Verified, registered for subservice, GPS within 30km, offline allowed (push notification alerted).	180 Seconds	No nearby partner responds.
TIER 3: BROADCAST	Any verified partner enrolled in the service category within city boundaries.	Configurable	Admin alert if unassigned timeout triggers.

Partner Scoring Algorithm

Eligible partners are ranked dynamically using the multi-factor scoring formula:

> `Score = (DistanceScore × 0.40) + (RatingScore × 0.30) + (AcceptanceRate × 0.20) - (CancellationPenalty × 0.10)`
>
> DistanceScore decreases linearly from 1.0 (at 0km) to 0.0 (at 30km). High-rated partners with reliable completion histories receive first-look preference.

Lead Fee Hold & Acceptance Concurrency

1. **Wallet Credit Validation**
   Before offering a job request, `leadHoldService` verifies: `(wallet_balance - reserved_balance + credit_limit) ≥ lead_fee`. If true, the lead fee is placed in `reserved_balance`.

2. **Concurrency & Race Condition Prevention**
   Acceptance executes via atomic SQL: `UPDATE provider.job_requests SET status = 'accepted' WHERE id = $1 AND status = 'pending' RETURNING *`. Exactly one partner can claim the job; competing requests are cancelled immediately.

3. **Automated Reconciliation Sweep**
   The `reconciliationWorker` runs every 60 seconds to expire stale job requests, automatically unfreeze reserved wallet balances, and set stale-GPS partners to offline.

5. Payment Escrow, Execution & Settlement Workflow

BharatClap enforces an **escrow custody model**: funds paid by the customer remain held securely in the escrow account until work completion is confirmed via dual-stage customer OTPs.

> **Dual-Stage Verification OTPs**
>
> **Start OTP:** Verified when partner reaches customer premises. Prevents phantom visits.
> **Completion OTP:** Given by customer only when satisfied. Triggers booking completion and unlocks partner settlement.

Step-by-Step Financial Flow

1. **Order Creation & Escrow Collection**
   Customer triggers checkout → `payment-service` creates an order with Razorpay → Customer completes payment. Webhook `payment.captured` is validated via HMAC-SHA256 signature and stored with idempotency protection in `payment.payments`.

2. **Job Execution Verification**
   Partner marks `arrived` → Enters Start OTP → Service in progress → Finishes work → Customer shares Completion OTP → Booking status transitions to `completed`.

3. **Settlement Computation & Platform Commission**
   Booking completion emits an outbox event to `provider-service`. The settlement calculator computes: `Partner Payout = Booking Total - Platform Commission (e.g., 15%) - Lead Fee Adjustments + Tips`. A record is created in `provider.provider_settlements` with status `ready_for_payout`.

4. **Batch Settlement & RazorpayX Disbursal**
   The `batchSettlementProcessor` claims eligible records with an atomic lock (`status = 'processing'`). It retrieves and decrypts partner bank details, generates a deterministic idempotency key (`settlement_{id}_attempt_{n}`), and calls RazorpayX Payouts.

5. **Double-Entry Ledger & Dispute Safeguards**
   Once confirmed by RazorpayX, the provider's wallet ledger is credited/settled, and balanced journal entries are recorded in `accounting.journal_entries`. If a dispute or refund request is logged before payout, the settlement engine halts processing and routes to `refund_pending`.

Financial Integrity & Audit Matrix

COMPONENT	FAILURE SCENARIO	AUTOMATED DEFENSE MECHANISM
Razorpay Webhook	Duplicate network retries	Unique constraint on `razorpay_payment_id`; duplicate events return HTTP 200 without reprocessing.
Settlement Payout	Gateway timeout mid-transfer	Deterministic idempotency key ensures RazorpayX never performs duplicate bank transfers on retries.
Customer Refund	Refund requested while payout processing	Settlement claim query explicitly checks `refund_event_id IS NULL`; flags `refund_pending` immediately.
Wallet Mutation	Concurrent balance deduction	Cryptographic hash chaining, row-level locks (`FOR UPDATE`), and DB immutability triggers.

6. Microservices Technical Directory

The complete codebase is organized across independent workspaces, each with distinct build pipelines, configuration baselines, and test suites:

SERVICE / APP	LOCATION	PORT	PRIMARY RESPONSIBILITIES
API Gateway	`/api-gateway`	`5000`	Reverse proxy, rate limiting, helmet security, JWT verification, CORS policy.
Auth Service	`/auth-service`	`5001`	User & provider onboarding, Fast2SMS OTP, sessions, token rotation, RBAC.
Catalog Service	`/catalog-service`	`5002`	Categories, subservices, dynamic pricing rules, starter kits, banner storage.
Provider Service	`/provider-service`	`5003`	PostGIS matching, scoring engine, lead hold, wallet ledger, RazorpayX payouts.
Booking Service	`/booking-service`	`5004`	Booking state machine, pricing engine, 3-tier dispatch queue, event outbox.
Payment Service	`/payment-service`	`5005`	Razorpay orders, webhook signature verification, escrow holding, refunds.
Notification Service	`/notification-service`	`5006`	FCM push notifications, transactional SMS, scheduled booking cron workers.
Web Frontend	`/frontend`	`3000`	Next.js App Router, TailwindCSS, React Query, customer booking flow.
Mobile Apps	`bharatclapapp/`	Mobile	Customer App & Service Partner App (GPS tracking, socket job alerts).

> **Platform Production Readiness**
>
> All services comply with the **Phase 1 Production Runbook**: featuring PostgreSQL connection pooling, health probes (`/health`), distributed tracing correlation IDs, and automated zero-downtime database migration runners.